

每日減脂計畫 App 規格書

2026-09-22 · @Someone

產品概述

每日減脂計畫是一個 iPhone／Android App，用「今日流程」把量身形、三餐和運動串成每天照著走的步驟，幫使用者養成減脂習慣。

項目	內容
目標	每天完成流程：起床量體重和腰圍 → 早餐 → 午餐 → 運動 → 晚餐
第一位使用者	31 歲女性、164 cm、56 kg、久坐，每日目標 1,340 大卡、蛋白質 101 g
平台	原生 App (Expo／React Native)，後端自寫 API 伺服器 + SQLite
餐點來源	使用者自建餐點，由食物分類組成，可等量替換
訓練	在家與健身房兩套課表，可自行編輯

相關連結：

- [Wireframe 設計畫布](#)：15 個畫面，每個畫面旁有對應的 API
- [網頁版原型](#)：已可每日使用，用來驗證流程和餐點設計

功能需求

原始 spec 的每一項都已對應到 wireframe 畫面；拍照辨識排在第三階段開發。

編號	需求	對應畫面	設計狀態
1a	每天記錄體重和腰圍	今日流程第 1 步、身形追蹤	完成
1b	圖表	身形追蹤（體重、腰圍趨勢，7 天平均）	完成
2a	建立、檢視餐點，用克數計算營養素和熱量	我的餐點、編輯餐點、食物庫、加入食物	完成

編號	需求	對應畫面	設計狀態
2b	拍照辨識餐點裡的食物	拍照辨識、辨識結果	完成，第三階段開發
2c-i	使用者新增自己的餐點	編輯餐點（依分類組成）	完成
2c-ii	隨機分配到早午晚餐	我的餐點（每日自動分配開關）	完成
3a	建立、編輯、檢視訓練動作（組數、次數）	訓練、編輯課表	完成
4a	早上先量體重和腰圍，再開始早餐	今日流程	完成
4b	早餐從使用者自己的菜單推薦	今日流程	完成
4c	午晚餐同樣流程	今日流程	完成
4d	運動 checklist	今日流程（逐組打勾）	完成

核心功能（第一版）

第一版做五大塊：帳號與設定、身形追蹤、餐點、訓練、今日流程；AI 與進階功能延後。

0. 帳號與個人設定

- 註冊、登入（Email、Google、Apple）、登出、刪除帳號（App Store 必要條件）
- 個人資料：性別、年齡、身高、體重、活動量、減脂速度
- 自動算出每日熱量與三大營養素目標，資料修改後重算
- 主食份量隨目標自動調整（開關，預設開啟）

1. 身形追蹤

- 每天記錄體重和腰圍，可以只填一個；可修改、刪除
- 體重、腰圍趨勢圖，附 7 天平均線
- 本週平均與上週相比的變化

2. 餐點

- 食物庫：依分類瀏覽和搜尋，每樣食物有每 100 g 營養素和常用份量
- 自建餐點：依分類加入食物、輸入份量，自動計算熱量和營養素
- 等量替換：同分類食物互換時自動換算克數，顯示營養差異和份量上限
- 每道餐點設定適合的時段（早餐、午餐、晚餐）和類型（日常、清淡、偶爾吃）
- 每日自動分配：從使用者的「日常」餐點整道挑選，午晚餐不重複

3. 訓練

- 動作：名稱、組數、次數或時間、重量、做法說明
- 課表：組合動作、調整順序，分在家和健身房
- 一週排程：星期幾做哪份課表

4. 今日流程

- 一次只顯示一步，完成後才進入下一步，不需要捲動
- 進度列顯示 5 個步驟，可點已完成的步驟回頭修改
- 運動可選早餐後或晚餐前，步驟順序跟著改變
- 餐點可只替換其中一樣食物，或整道換掉
- 完成畫面顯示當天總結

延後的功能：拍照辨識（第三階段）、文字記錄飲食、Jev 與其他 AI 功能、每週回顧、推播提醒、點心日、計畫外食物記錄、訓練強度分級、步數、休息計時器、連續天數。若第一版就上架原生 App，建議把「每天早上提醒量體重」的推播移進第一版。

畫面清單

共 15 個畫面，底部分頁為今天、身形、餐點、訓練、設定；完整設計見 [wireframe 畫布](#)。

區塊	畫面	重點內容與互動
帳號	登入	「使用 Google 繼續」「使用 Apple 繼續」；下方為 Email、密碼、忘記密碼、註冊連結
帳號	建立個人資料	性別、年齡、身高、體重、腰圍（選填）、活動量、減脂速度；即時顯示每日目標
帳號	設定	個人資料、每日目標、運動時間預設、早上提醒、主食份量自動調整、登出、刪除帳號
今日流程	今日流程	進度列 5 步；一次一步；底部大按鈕寫明下一步；餐點逐項「換」或「整道換掉」；完成畫面
身形	身形追蹤	輸入體重和腰圍、本週平均與變化、兩張趨勢圖、紀錄列表
餐點	我的餐點	「我的餐點／食物庫」切換、時段篩選、自動分配開關、餐點列出分類組成、相機按鈕
餐點	食物庫	依分類瀏覽，顯示營養素和常用份量、換算依據
餐點	編輯餐點	名稱、時段、類型；依分類分區，每樣可改份量、替換、移除；拍照加入；合計即時重算

區塊	畫面	重點內容與互動
餐點	替換食物	目前食物；等蛋白質／等熱量；同分類選項顯示換算克數、營養差異、上限提示
餐點	加入食物	搜尋、分類篩選、相機按鈕；點整列選取；下方設定份量後加入
餐點	拍照辨識	圓形取景框、相簿、快門；標示要加入哪道餐點
餐點	辨識結果	依分類列出食物與估計克數；信心高／中／請確認；找不到的可選擇或略過；三種儲存方式
訓練	訓練	在家／健身房分頁、一週排程、課表列表
訓練	編輯課表	名稱、地點、時間；動作的組數、次數、重量、說明；排序、新增
今日流程	換動作	選擇原因（器材被佔用、膝蓋不適、沒有這個器材、想換換口味）；Jev 排序的同分類替代動作，第一個標「最推薦」；只換今天

共用元件 FoodOptionRow：用於加入食物、替換食物（之後也可用於辨識結果的候選）。點整列選取，選中時淡主色背景；右側固定保留 24 px 的 ✓ 位置，選取只切換 ✓ 是否顯示，不造成版面位移。替換頁的克數緊接在食物名稱右側。

營養與計算規則

熱量和營養素一律由程式依食物庫計算，不交給 AI；克數可以是系統建議值，但使用者永遠能修改。

每日目標（Mifflin-St Jeor 公式）

$$\text{BMR} = 10 \times W + 6.25 \times H - 5 \times A + s$$

W 為體重 (kg)、H 為身高 (cm)、A 為年齡；s 女性為 -161、男性為 +5。每日總消耗 = BMR × 活動係數（久坐 1.2、每週運動 1-3 天 1.375、3-5 天 1.55、6-7 天 1.725）。

項目	規則
熱量目標	每日總消耗 × (1 - 減脂速度 10/12/15/20%)，不低於基礎代謝，女性最低 1,200 大卡
蛋白質	體重 × 1.8 g
脂肪	體重 × 0.8 g

項目	規則
碳水	剩餘熱量 ÷ 4
範例	164 cm、56 kg、31 歲女性、久坐、12%：BMR 1,269，總消耗 1,523，目標 1,340 大卡、蛋白質 101 g、脂肪 45 g、碳水 133 g

克數從哪裡來

情境	克數來源
從食物庫加入	預設帶入常用份量 (usual_grams)
替換食物	等量換算自動計算
拍照辨識	AI 估計，使用者確認後才儲存
編輯時修改	使用者輸入

等量替換：依分類的換算依據（主食看碳水、蛋白質類看蛋白質，也可切換為等熱量）。

$$\text{新克數} = \text{原克數} \times \frac{\text{原食物每 100 g 營養素}}{\text{新食物每 100 g 營養素}}$$

例：雞胸肉 100 g 換雞腿， $100 \times 31 \div 24 \approx 129$ ，取 130 g。一般食物取到 5 或 10 g，以顆、匙計的取整數；超過 max_grams 時只換到上限，並顯示營養素差多少。

主食份量自動調整：餐點存的是基準克數。熱量目標改變時，當天計畫中的主食克數乘上 carb_scale (= 新碳水目標 ÷ 基準碳水目標)，蛋白質和蔬菜不變；編輯頁仍顯示基準克數。

資料模型與 seed

資料庫共 23 張表，分成三群：帳號與追蹤、食物與餐點、每日紀錄與訓練，另有一張共用的 translations 翻譯表；完整結構就是第一個 migration 檔 migrations/0001_init.sql。

1. 帳號與追蹤

```
erDiagram
    users ||--|| profiles : "個人資料"
    users ||--o{ user_identities : "Google、Apple 登入"
    users ||--o{ refresh_tokens : "登入"
    users ||--o{ devices : "推播"
    users ||--o{ body_logs : "每日身形"
    users ||--o{ meal_photos : "拍照"
```

2. 食物與餐點

```
erDiagram
    food_categories ||--o{ foods : "分類"
    foods ||--o{ food_aliases : "別名"
    meals ||--o{ meal_times : "適合時段"
    meals ||--o{ meal_items : "組成"
    foods ||--o{ meal_items : "食物"
```

3. 每日紀錄與訓練

```
erDiagram
    days ||--o{ day_meals : "早午晚餐與 extras"
    day_meals ||--o{ day_meal_items : "當天實際吃的"
    days ||--o{ set_logs : "打勾的組數"
    exercise_categories ||--o{ exercises : "分類"
    exercise_categories ||--o{ workout_templates : "分類"
    workout_templates ||--o{ workout_template_items : "動作"
    exercises ||--o{ workout_template_items : "引用"
    workout_templates ||--o{ workout_schedule : "一週排程"
    exercises ||--o{ set_logs : "重量歷史"
```

設計原則

- **主鍵**：UUID v7（Python 端產生，依時間排序）；以「使用者＋日期」為單位的表用複合主鍵，例如 body_logs 的 (user_id, date)
- **時間與日期**：時間戳記存 UTC 的 ISO 8601 文字；日期存使用者時區的 YYYY-MM-DD
- **多語系**：內建資料（食物、分類、動作）只存 name_key，文字放在 translations，依使用者語系取出，找不到時退回 zh-TW；使用者自建的資料直接存 name，不翻譯
- **內建與自訂**：foods、exercises 的 user_id 為 NULL 代表內建，使用者不能修改
- **分類一致**：食物用 food_categories、動作與課表用 exercise_categories，兩者結構相同
- **刪除帳號**：所有使用者資料透過 ON DELETE CASCADE 一次刪除；MinIO 上的照片由刪除流程依 meal_photos.object_key 一併清除
- **軟刪除**：meals、foods、exercises、workout_templates 用 archived_at，歷史紀錄仍能引用
- **每日快照**：day_meal_items 記錄當天實際的食物和克數（已套用替換與 carb_scale），之後修改餐點不會改變過去的紀錄
- **額外食物**：不另外開表，當成 day_meals 的第四個時段 extras；只知道熱量的項目也能記（food_id 為 NULL，改填 custom_name 與營養素）
- **分類不重複存**：食物分類只存在 foods.category_id，餐點項目透過食物取得分類
- **自製料理**：不另外建模，自製料理就是一道餐點；例如番茄蛋花湯的番茄、金針菇、蛋直接放進 meal_items

- **列舉值**：用 CHECK 限制，例如 meal_time 只能是 breakfast、lunch、dinner、extras

完整結構 (migrations/0001_init.sql)

```
-- ===== 共用 =====
CREATE TABLE translations (
  key      TEXT NOT NULL,          -- 例如
  food.chicken_thigh_cooked.name
  locale   TEXT NOT NULL,         -- zh-TW、en
  text     TEXT NOT NULL,
  PRIMARY KEY (key, locale)
) WITHOUT ROWID;

-- ===== 帳號與追蹤 =====
CREATE TABLE users (
  id          TEXT PRIMARY KEY,
  email       TEXT NOT NULL UNIQUE COLLATE NOCASE,
  password_hash TEXT,             -- 只用 Google／Apple 登入的
  使用者為 NULL
  locale      TEXT NOT NULL DEFAULT 'zh-TW',
  created_at  TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now'))
);

CREATE TABLE refresh_tokens (
  id          TEXT PRIMARY KEY,
  user_id     TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  token_hash  TEXT NOT NULL UNIQUE,
  expires_at  TEXT NOT NULL,
  revoked_at  TEXT,
  created_at  TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now'))
);
CREATE INDEX idx_refresh_tokens_user ON refresh_tokens(user_id);

CREATE TABLE user_identities (
  provider     TEXT NOT NULL CHECK (provider IN ('google', 'apple')),
  provider_subject TEXT NOT NULL,  -- Google／Apple token 裡
  的 sub
  user_id      TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  email        TEXT,              -- 登入當下取得的 email，僅
  供參考
  created_at   TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ',
'now')),
  PRIMARY KEY (provider, provider_subject)
) WITHOUT ROWID;
CREATE INDEX idx_user_identities_user ON user_identities(user_id);

CREATE TABLE profiles (
```

```

user_id      TEXT PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
sex          TEXT NOT NULL CHECK (sex IN ('f', 'm')),
birth_date   TEXT NOT NULL,                      -- 年齡由此計算
height_cm    REAL NOT NULL CHECK (height_cm BETWEEN 100 AND 250),
weight_kg     REAL NOT NULL CHECK (weight_kg BETWEEN 30 AND 300),
activity_level TEXT NOT NULL CHECK (activity_level IN ('sedentary',
'light', 'moderate', 'active')),
deficit_pct   INTEGER NOT NULL CHECK (deficit_pct IN (10, 12, 15, 20)),
carb_base_g   REAL NOT NULL,                      -- 餐點基準份量對應的碳水目標
auto_scale_carbs INTEGER NOT NULL DEFAULT 1 CHECK (auto_scale_carbs IN (0,
1)),      -- 設定：主食份量隨目標自動調整
auto_assign_meals INTEGER NOT NULL DEFAULT 1 CHECK (auto_assign_meals IN
(0, 1)),  -- 我的餐點：每日自動分配開關
workout_time  TEXT NOT NULL DEFAULT 'pm' CHECK (workout_time IN ('am',
'pm')),
default_location TEXT NOT NULL DEFAULT 'home' CHECK (default_location IN
('home', 'gym')), -- 平常運動地點，決定預設課表
reminder_time  TEXT,                             -- 'HH:MM', NULL 代表不提醒
timezone       TEXT NOT NULL DEFAULT 'Australia/Brisbane',
updated_at     TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ',
'now'))
);

```

```

CREATE TABLE devices (
  id          TEXT PRIMARY KEY,
  user_id     TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  push_token  TEXT NOT NULL UNIQUE,
  platform    TEXT NOT NULL CHECK (platform IN ('ios', 'android')),
  last_seen_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now'))
);
CREATE INDEX idx_devices_user ON devices(user_id);

```

```

CREATE TABLE body_logs (
  user_id     TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  date        TEXT NOT NULL,
  weight_kg   REAL CHECK (weight_kg BETWEEN 30 AND 300),
  waist_cm    REAL CHECK (waist_cm BETWEEN 40 AND 200),
  updated_at  TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),
  PRIMARY KEY (user_id, date),
  CHECK (weight_kg IS NOT NULL OR waist_cm IS NOT NULL)
) WITHOUT ROWID;

```

```

CREATE TABLE meal_photos (
  id          TEXT PRIMARY KEY,
  user_id     TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  object_key  TEXT NOT NULL,                      -- MinIO 物件路徑 meal-

```

```

photos/{user_id}/{id}.jpg
  status      TEXT NOT NULL DEFAULT 'uploaded' CHECK (status IN ('uploaded',
'analyzing', 'done', 'failed')),
  result_json TEXT,                                -- analyze 的原始回傳
  created_at  TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),
  analyzed_at TEXT
);
CREATE INDEX idx_meal_photos_user ON meal_photos(user_id, created_at);

-- ===== 食物與餐點 =====
CREATE TABLE food_categories (
  id          TEXT PRIMARY KEY,                    -- staple, protein,
vegetable, fruit
  name_key    TEXT NOT NULL,                      --
food_category.protein.name
  swap_by     TEXT NOT NULL CHECK (swap_by IN ('carb', 'protein', 'kcal',
'none')), -- 替換時保持相同的營養素
  sort_order  INTEGER NOT NULL
);

CREATE TABLE foods (
  id          TEXT PRIMARY KEY,
  user_id     TEXT REFERENCES users(id) ON DELETE CASCADE, -- NULL 代
表內建
  category_id TEXT NOT NULL REFERENCES food_categories(id),
  name_key    TEXT,                                -- 內建食物
用
  name        TEXT,                                -- 使用者自
建食物用
  state       TEXT NOT NULL DEFAULT 'na' CHECK (state IN ('raw',
'cooked', 'na')),
  kcal_per_100g REAL NOT NULL CHECK (kcal_per_100g >= 0),
  protein_per_100g REAL NOT NULL CHECK (protein_per_100g >= 0),
  fat_per_100g   REAL NOT NULL CHECK (fat_per_100g >= 0),
  carb_per_100g  REAL NOT NULL CHECK (carb_per_100g >= 0),
  fiber_per_100g REAL CHECK (fiber_per_100g >= 0),
  unit          TEXT NOT NULL DEFAULT 'g' CHECK (unit IN ('g', 'ml',
'piece', 'scoop', 'bowl')),
  grams_per_unit REAL CHECK (grams_per_unit > 0), -- 雞蛋 50、乳清 30、豆漿
1.03
  usual_grams   REAL NOT NULL CHECK (usual_grams > 0),
  max_grams     REAL NOT NULL,
  source        TEXT CHECK (source IN ('AFCD', 'TFDA', 'label', 'user')),
  source_id     TEXT,
  archived_at   TEXT,
  created_at    TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ',

```

```

'now')),
  updated_at      TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ',
'now')),
  CHECK ((name_key IS NULL) <> (name IS NULL)),
  CHECK (max_grams >= usual_grams),
  CHECK (unit = 'g' OR grams_per_unit IS NOT NULL)
);
CREATE INDEX idx_foods_category ON foods(category_id) WHERE archived_at IS
NULL;
CREATE INDEX idx_foods_user ON foods(user_id);

CREATE TABLE food_aliases (                                -- 搜尋與辨識用的同義詞，不是翻
譯
  food_id TEXT NOT NULL REFERENCES foods(id) ON DELETE CASCADE,
  alias   TEXT NOT NULL COLLATE NOCASE,                    -- 雞腿、乾煎雞腿、chicken
thigh
  PRIMARY KEY (food_id, alias)
) WITHOUT ROWID;
CREATE INDEX idx_food_aliases_alias ON food_aliases(alias);

CREATE TABLE meals (
  id          TEXT PRIMARY KEY,
  user_id     TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  name        TEXT NOT NULL,
  tag         TEXT NOT NULL DEFAULT 'regular' CHECK (tag IN ('regular',
'light', 'occasional')),
  archived_at TEXT,
  created_at  TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),
  updated_at  TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now'))
);
CREATE INDEX idx_meals_user ON meals(user_id) WHERE archived_at IS NULL;

CREATE TABLE meal_times (
  meal_id     TEXT NOT NULL REFERENCES meals(id) ON DELETE CASCADE,
  meal_time   TEXT NOT NULL CHECK (meal_time IN ('breakfast', 'lunch',
'dinner')),
  PRIMARY KEY (meal_id, meal_time)
) WITHOUT ROWID;

CREATE TABLE meal_items (                                -- 餐點的組成 (範本)
  id          TEXT PRIMARY KEY,
  meal_id     TEXT NOT NULL REFERENCES meals(id) ON DELETE CASCADE,
  food_id     TEXT NOT NULL REFERENCES foods(id),
  grams       REAL NOT NULL CHECK (grams > 0),            -- 基準克數
  sort_order  INTEGER NOT NULL DEFAULT 0
);

```

```

CREATE INDEX idx_meal_items_meal ON meal_items(meal_id, sort_order);

-- ===== 訓練 =====
CREATE TABLE exercise_categories (
  id          TEXT PRIMARY KEY,                -- strength, cardio,
  mobility
  name_key    TEXT NOT NULL,
  sort_order  INTEGER NOT NULL
);

CREATE TABLE exercises (
  id          TEXT PRIMARY KEY,
  user_id     TEXT REFERENCES users(id) ON DELETE CASCADE,  -- NULL 代表
內建
  category_id TEXT NOT NULL REFERENCES exercise_categories(id),
  name_key    TEXT,                                         -- 內建動作用
  name        TEXT,                                         -- 使用者自建
動作用
  description_key TEXT,
  description  TEXT,
  archived_at  TEXT,
  created_at   TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ',
'now')),
  CHECK ((name_key IS NULL) <> (name IS NULL))
);

CREATE TABLE workout_templates (
  id          TEXT PRIMARY KEY,
  user_id     TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  category_id TEXT NOT NULL REFERENCES exercise_categories(id),
  name        TEXT NOT NULL,
  location    TEXT NOT NULL CHECK (location IN ('home', 'gym', 'both')),
  duration_min INTEGER CHECK (duration_min BETWEEN 0 AND 240),
  archived_at TEXT,
  created_at  TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),
  updated_at  TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now'))
);

CREATE INDEX idx_workout_templates_user ON workout_templates(user_id) WHERE
archived_at IS NULL;

CREATE TABLE workout_template_items (
  id          TEXT PRIMARY KEY,
  template_id TEXT NOT NULL REFERENCES workout_templates(id) ON DELETE
CASCADE,
  exercise_id TEXT NOT NULL REFERENCES exercises(id),
  sort_order  INTEGER NOT NULL,

```

```

    sets          INTEGER CHECK (sets BETWEEN 1 AND 10),  -- NULL 代表不分組 (暖身、
有氧)
    reps          TEXT NOT NULL,                          -- 「10-12 下」「30 秒」
    weight_kg     REAL CHECK (weight_kg >= 0),
    rest_sec     INTEGER NOT NULL DEFAULT 60,
    note         TEXT
);
CREATE INDEX idx_template_items ON workout_template_items(template_id,
sort_order);

CREATE TABLE workout_schedule (
    user_id       TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    weekday      INTEGER NOT NULL CHECK (weekday BETWEEN 0 AND 6),  -- 0 = 週日
    location     TEXT NOT NULL CHECK (location IN ('home', 'gym')),
    template_id  TEXT NOT NULL REFERENCES workout_templates(id),
    PRIMARY KEY (user_id, weekday, location)
) WITHOUT ROWID;

-- ===== 每日紀錄 =====
CREATE TABLE days (
    user_id       TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    date         TEXT NOT NULL,
    workout_time  TEXT NOT NULL CHECK (workout_time IN ('am', 'pm')),
    location     TEXT NOT NULL CHECK (location IN ('home', 'gym')),
    template_id  TEXT REFERENCES workout_templates(id) ON DELETE SET NULL,
    carb_scale   REAL NOT NULL DEFAULT 1.0,                -- 當天套用的主食倍率
    steps        INTEGER CHECK (steps >= 0),
    workout_done_at TEXT,
    created_at   TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ',
'now')),
    PRIMARY KEY (user_id, date)
) WITHOUT ROWID;

CREATE TABLE day_meals (                                -- 當天每個時段吃哪一道
    user_id     TEXT NOT NULL,
    date       TEXT NOT NULL,
    meal_time  TEXT NOT NULL CHECK (meal_time IN ('breakfast', 'lunch',
'dinner', 'extras')),
    meal_id    TEXT REFERENCES meals(id),                -- 來源餐點; extras 為
NULL
    eaten_at  TEXT,                                     -- NULL 代表還沒吃
    PRIMARY KEY (user_id, date, meal_time),
    FOREIGN KEY (user_id, date) REFERENCES days(user_id, date) ON DELETE
CASCADE,
    CHECK (meal_time <> 'extras' OR meal_id IS NULL)
) WITHOUT ROWID;

```

```

CREATE TABLE day_meal_items (
    id TEXT PRIMARY KEY,
    user_id TEXT NOT NULL,
    date TEXT NOT NULL,
    meal_time TEXT NOT NULL,
    food_id TEXT REFERENCES foods(id),
    custom_name TEXT,
    grams REAL CHECK (grams > 0),
    kcal REAL CHECK (kcal >= 0),
    protein_g REAL CHECK (protein_g >= 0),
    fat_g REAL CHECK (fat_g >= 0),
    carb_g REAL CHECK (carb_g >= 0),
    photo_id TEXT REFERENCES meal_photos(id) ON DELETE SET NULL,
    sort_order INTEGER NOT NULL DEFAULT 0,
    created_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),
    FOREIGN KEY (user_id, date, meal_time)
        REFERENCES day_meals(user_id, date, meal_time) ON DELETE CASCADE,
    CHECK ((food_id IS NOT NULL AND grams IS NOT NULL)
        OR (food_id IS NULL AND custom_name IS NOT NULL AND kcal IS NOT NULL))
);
CREATE INDEX idx_day_meal_items_slot ON day_meal_items(user_id, date, meal_time);

CREATE TABLE set_logs (
    id TEXT PRIMARY KEY,
    user_id TEXT NOT NULL,
    date TEXT NOT NULL,
    template_item_id TEXT REFERENCES workout_template_items(id) ON DELETE SET NULL,
    exercise_id TEXT NOT NULL REFERENCES exercises(id),
    set_index INTEGER NOT NULL CHECK (set_index >= 0),
    reps_done INTEGER CHECK (reps_done >= 0),
    weight_kg REAL CHECK (weight_kg >= 0),
    done_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now')),
    UNIQUE (user_id, date, template_item_id, set_index),
    FOREIGN KEY (user_id, date) REFERENCES days(user_id, date) ON DELETE CASCADE
);
CREATE INDEX idx_set_logs_history ON set_logs(user_id, exercise_id, date);

```

今日流程的「目前第幾步」不另外存，由 body_logs、day_meals.eaten_at 與 days.workout_done_at 推算；連續天數也由 days 推算。

foods 的 API 回應格式（資料庫中營養素是各自的欄位）

```
{
  "id": "chicken-thigh-skin-on-cooked",
  "name": "去骨雞腿（帶皮、熟）",
  "aliases": ["雞腿", "乾煎雞腿", "chicken thigh"],
  "category": "protein",
  "state": "cooked",
  "per_100g": { "kcal": 210, "protein": 24, "fat": 13, "carb": 0 },
  "unit": "g",
  "usual_grams": 100,
  "max_grams": 200,
  "source": "AFCD"
}
```

Seed 資料

Seed	內容	資料來源
食物分類	主食、蛋白質、蔬菜、水果、油脂與醬料，及換算依據	自訂
食物	先人工整理 100-150 種常見食物	Australian Food Composition Database (FSANZ)、台灣衛福部食品營養成分資料庫、品牌包裝標示
訓練動作	在家與健身房常用動作，附做法說明	自訂
範例餐點與課表（選用）	讓新使用者一進來就能開始今日流程	自訂

注意事項

- state 標清楚生或熟：糙米生重與熟重熱量差近 3 倍
- 以顆、匙、ml 計的食物填 grams_per_unit，例如雞蛋 50 g／顆
- seed 用 id 做「有就更新、沒有就新增」，可重複執行
- 使用者不能改內建食物，但可複製一份再改
- 上線前確認資料庫的授權條款允許用於 App

API 規格

REST + JSON，登入後帶 `Authorization: Bearer <token>`；日期 YYYY-MM-DD（使用者時區），單位 g、kg、cm、kcal。階段：P1 核心、P2 餐點、P3 拍照辨識。

方法	路徑	用途	階段
POST	/auth/register、/auth/login、/auth/refresh、/auth/logout	帳號	P1
DELETE	/users/me	刪除帳號和所有資料	P1
GET、PATCH	/users/me/profile	個人資料與偏好（含 auto_scale_carbs）	P1
GET	/users/me/targets	基礎代謝、總消耗、熱量與營養素目標、carb_scale	P1
GET	/body-logs?from=&to=	身形紀錄列表	P1
PUT、DELETE	/body-logs/{date}	新增或更新、刪除某天紀錄	P1
GET	/body-logs/summary?from=&to=	圖表資料、7 天平均、週平均與變化	P1
GET	/days/{date}	今日流程狀態：目前步驟、已吃熱量、目標	P1
PATCH	/days/{date}	運動時間、地點、步數	P1
PATCH	/days/{date}/meals/{meal_time}	標記吃完	P1
PUT	/days/{date}/workout/sets	打勾一組，可記實際次數與重量	P1
GET、POST	/exercises	動作庫、新增自訂動作	P1
PATCH、DELETE	/exercises/{id}	修改、刪除自訂動作	P1
GET、POST	/workout-templates?location=	課表列表、新增課表	P1
GET、PATCH、DELETE	/workout-templates/{id}	單一課表	P1
POST、PATCH、DELETE	/workout-templates/{id}/items/{itemId}	課表中的動作	P1
PUT	/workout-templates/{id}/items/order	調整順序	P1
GET、PUT	/workout-schedule	一週排程	P1
GET	/food-categories	分類與換算依據	P2
GET	/foods?q=&category=	搜尋、依分類列出食物	P2
POST、PATCH、DELETE	/foods/{id}	自訂食物、常用份量	P2

方法	路徑	用途	階段
GET	/foods/{id}/exchanges?grams=&match=	同分類所有食物的換算克數、營養差異、是否到上限	P2
GET、POST	/meals?meal_time=&q=	餐點列表、新增	P2
GET、PATCH、DELETE	/meals/{id}	單一餐點；items 為 { category, food_id, grams }	P2
POST	/meals/calculate	不儲存的即時試算	P2
GET	/days/{date}/plan	當天三餐與課表，主食克數已依 carb_scale 調整；沒有就隨機產生	P2
POST	/days/{date}/plan/shuffle	整道換掉，可指定 meal_time	P2
PATCH	/days/{date}/plan/{meal_time}/items	換某一樣 { category, to_food_id }，自動等量換算	P2
POST	/meal-photos	取得照片上傳網址	P3
POST	/meal-photos/{id}/analyze	辨識食物並對應食物庫	P3
POST、DELETE	/days/{date}/meals/extras/items[/{id}]	只記錄當天吃的額外食物	P3
POST	/devices、PATCH /users/me/reminders	推播 token、提醒時間（原生 App）	P1
POST	/foods/suggest-category	Jev 依名稱建議食物分類，回傳 category_id 與信心	P3
POST	/exercises/suggest-category	Jev 依名稱建議動作分類	P3
POST	/meals/suggest-settings	Jev 依組成建議適合時段與類型	P3
GET	/exercises/{id}/alternatives?reason=	Jev 依原因挑出替代動作	P3
POST	/auth/google、/auth/apple	傳入 Google／Apple 的 ID token，驗證後回傳我們的 access 與 refresh token；新使用者自動建立帳號	P1

所有端點都由 FastAPI（Python）實作，資料存在 SQLite；計算類端點（targets、meals/calculate、foods/{id}/exchanges、days/{date}/plan、days/{date} 彙總）放在計算模組，meal-photos/analyze 放在 AI 模組。FastAPI 會自動產生 OpenAPI 文件（/docs），即為本節 API 的正式規格來源。

AI 功能

AI 只負責「辨識是什麼、大約幾克、選哪一個」，數字由程式計算，重要結果都由使用者確認。

拍照辨識流程

flowchart LR

```
A[App 拍照<br/>縮到長邊約 1024 px] --> B[上傳<br/>POST /meal-photos]
B --> C[圖片模型<br/>辨識食物與克數]
C --> D[Jev 從食物庫<br/>候選中挑選並給信心]
D --> E[使用者確認<br/>克數與食物]
E --> F[食物庫計算營養]
```

金鑰只放在後端；App 端不接觸任何 AI 金鑰。analyze 回傳每一樣：

```
{
  "label": "乾煎雞腿",
  "category": "蛋白質",
  "food_id": "chicken-thigh-skin-on",
  "grams": 110,
  "confidence": 0.62,
  "alternatives": ["pork-collar", "chicken-breast"]
}
```

confidence 決定畫面顯示「信心高」「信心中」或「請確認」；food_id 為 null 代表食物庫找不到，請使用者選擇或略過。

Jev 決策層：TypeSafe AI 的決策模型，回傳選擇、分數或是非判斷並附機率，2026 年 9 月起提供早期存取。

這個階段不做 chatbot，Jev 只放在現有流程中需要「判斷」的環節；計算、熱量上限、午晚餐不重複等硬規則一律由程式處理。

流程環節	畫面	Jev 決策	類型	API	信心不足時
拍照後對應食物	辨識結果	從食物庫候選（由 food_aliases 找出約 10 筆）挑出正確食物	選一個	analyze 內部	顯示「信心中」或「請確認」，列出其他可能
拍照後判斷份量	辨識結果	依圖片模型的描述（如「半碗」）從常用份量選項中擇一	選一個	analyze 內部	帶入常用份量，標示請確認
新增自訂食物	加入食物	依食物名稱建議分類	選一個	POST /foods/suggest-category	不預選，讓使用者自己挑
新增自訂動作	編輯課表	依動作名稱建議分類（肌力、有氧、伸展）	選一個	POST /exercises/suggest-category	不預選

流程環節	畫面	Jev 決策	類型	API	信心不足時
建立或編輯餐點	編輯餐點	依組成建議適合時段與類型（日常、清淡、偶爾吃）	是或否、選一個	POST /meals/suggest-settings	維持預設：午餐、晚餐，日常
每日自動分配	今日流程	從符合時段的日常餐點中，參考前幾天吃過什麼、今天是否為訓練日，挑出當天的一道	選一個	GET /days/{date}/plan 內部	退回隨機挑選
替換食物	替換食物	依使用者過去的選擇，排出最可能被選的替換選項	打分數	GET /foods/{id}/exchanges 內部	依熱量差距排序
動作做完後	今日流程運動步驟	依完成情況與使用者回饋（輕鬆、剛好、吃力），建議下次加重、維持或減輕	選一個	PUT /days/{date}/workout/sets 回傳	不給建議
換動作	今日流程運動步驟	依原因（器材被佔用、膝蓋不適、沒有啞鈴）從同分類動作中挑出替代	選一個	GET /exercises/{id}/alternatives? reason=	列出同分類動作讓使用者挑

對應的畫面：最後兩列已加入 wireframe。今日流程的運動步驟中，每個動作旁有「換動作」連結；動作做完後出現回饋按鈕（輕鬆、剛好、吃力）與下次重量建議，可一鍵「套用到課表」；「換動作」畫面列出原因與替代動作。

信心門檻：每個環節各自設定，例如食物對應 0.8 以上直接帶入、0.5-0.8 顯示「信心中」、0.5 以下請使用者確認；門檻用實際資料的測試集調整。所有 Jev 建議都只是預填或排序，使用者永遠可以改。

後端把決策層包成一個 decide() 函式，先用語言模型實作，之後可換成 Jev 比較速度、費用和準確度；上線前用自己的資料建立測試集驗證。

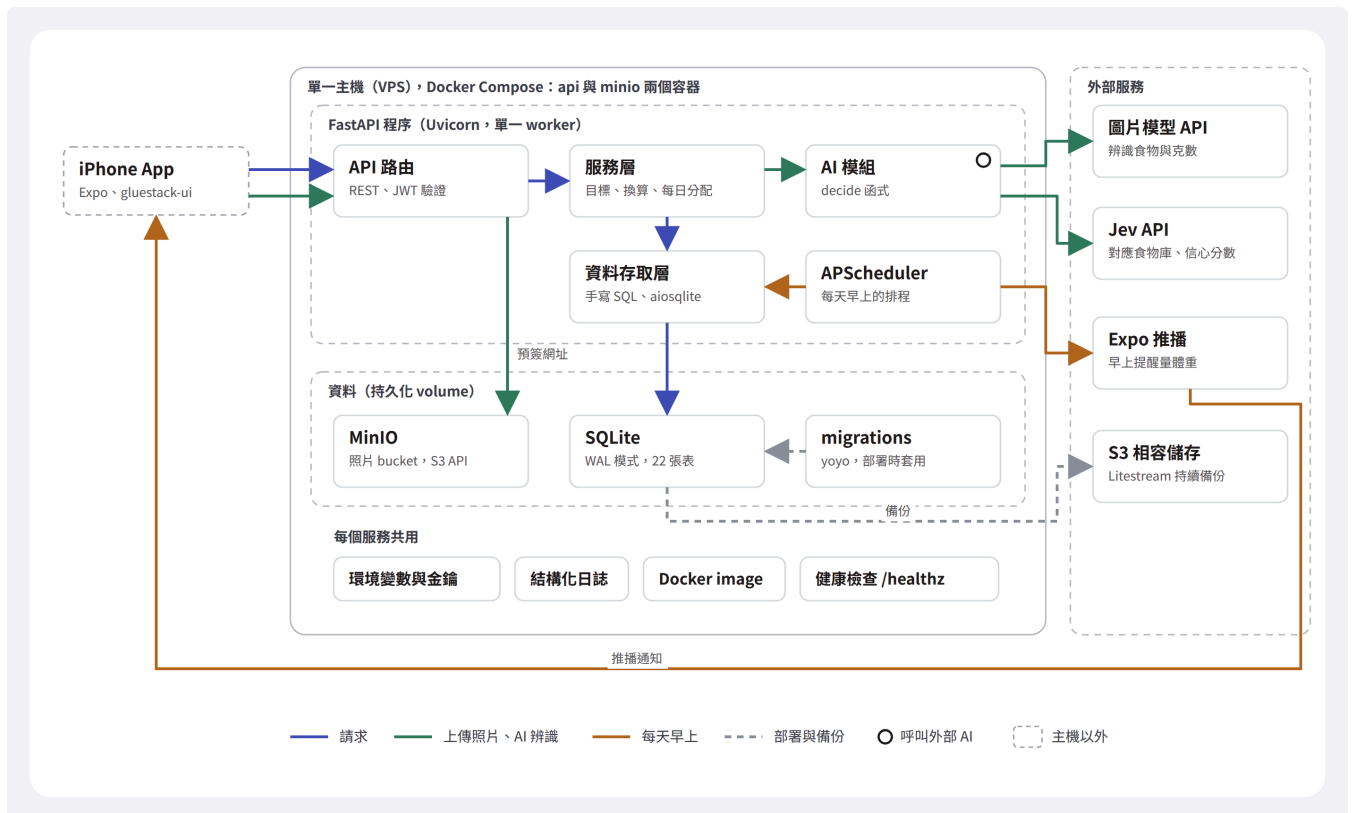
之後可加入的 AI 功能：文字或語音記錄飲食、每週回顧、依現有食材建議餐點、外食菜單建議、對話式教練。

安全原則

- 熱量建議不低於基礎代謝；每週減重不建議超過體重 1%
- 份量和辨識結果由使用者確認後才儲存
- 每位使用者每日呼叫上限；只傳必要資料，不傳姓名

技術架構與開發順序

第一版用單一主機：Expo App 透過 REST 呼叫自寫的 API 伺服器；結構化資料存在 SQLite，照片等檔案存在同一台主機上的 MinIO（S3 相容物件儲存）；AI 只在伺服器端呼叫。



藍線是一般請求：App → API 路由 → 服務層 → 資料存取層 → SQLite。綠線是拍照：App 向 API 取得預簽網址後直接把照片上傳到 MinIO，AI 模組再呼叫圖片模型與 Jev。橘線是每天早上：APScheduler 讀取提醒名單，透過 Expo 推播通知 App。灰色虛線是部署時套用 migrations，以及 Litestream 持續備份到主機以外的 S3 相容儲存。

App 只和 FastAPI 溝通；所有金鑰（圖片模型、Jev）都只在主機上。App 端的 TypeScript 型別和 API client 由 FastAPI 的 OpenAPI 規格自動產生（例如 openapi-typescript），後端改欄位時前端重新產生即可發現不一致。

元件	選擇	說明
前端	Expo + TypeScript	一套程式出 iPhone 與 Android；EAS Build 雲端打包，不需 Mac
UI 元件	gluestack-ui	按鈕、輸入框、切換器、彈窗等基礎元件；以複製方式放進專案，可直接改成 wireframe 的樣子
樣式	NativeWind (Tailwind 寫法)	顏色、字型、圓角集中在 tailwind.config.js，見下方設計 tokens
導覽	Expo Router	底部五個分頁與各頁面切換
圖表	Victory Native	身形追蹤的體重、腰圍趨勢圖
動畫	Reanimated	今日流程的步驟轉場

元件	選擇	說明
相機	expo-image-picker	拍照或從相簿選；輸出 JPEG，上傳前縮到長邊約 1024 px
API 型別	openapi-typescript	由 FastAPI 的 OpenAPI 規格產生
API 伺服器	FastAPI + Uvicorn	實作全部 API；Pydantic 驗證請求與回應，自動產生 OpenAPI 文件
登入	JWT (PyJWT) + argon2 密碼雜湊	access token 短效、refresh token 長效
資料存取	手寫 SQL + aiosqlite，不使用 ORM	每個資料表一個 repository 模組；一律用參數化查詢；查詢結果轉成 Pydantic 模型
Migrations	yoyo-migrations，純 .sql 檔	migrations/ 內依序編號，每個變更一個檔案加對應的 rollback 檔；部署時自動套用
資料庫	SQLite (WAL 模式)	每次連線設定 foreign_keys = ON、busy_timeout；放在持久化 volume
物件儲存	MinIO (S3 相容)，bucket：meal-photos	App 向 FastAPI 取得預簽網址後直接上傳，照片不經過 API；後端只用 S3 API (boto3)，日後可換其他 S3 相容服務而不改程式
Seed 匯入	Python 腳本 + INSERT ... ON CONFLICT DO UPDATE	讀 AFCD 等公開資料寫入，可重複執行；分類等固定資料放在 migration 中
排程	APScheduler	早上提醒等定時工作，與 API 同一個程序
備份	Litestream (SQLite) + rclone 定期同步 (MinIO)	目的地必須在主機以外，例如另一台主機或雲端的 S3 相容儲存
部署	單一 VPS + Docker Compose	api、minio 兩個容器，各自掛持久化 volume；SQLite 只能由一個伺服器實例寫入，Uvicorn 用單一 worker
推播	APScheduler + Expo 推播服務	每天早上提醒量體重
上架	Apple 開發者帳號每年 US\$99；Google Play 一次 US\$25	需隱私權政策、App 內刪除帳號、相機使用說明文字

元件	選擇	說明
第三方登入	App : react-native-nitro-google-signin、expo-apple-authentication ; 後端 : google-auth、PyJWT	需 development build (不能用 Expo Go) ; Google Cloud 建 Web、iOS、Android 三組 Client ID ; Apple token 以 Apple 公開金鑰驗證

MinIO 注意事項：官方社群版自 2025 年 10 月起不再提供執行檔與 Docker image，GitHub 庫已於 2026 年 4 月 25 日封存，不再有新功能或安全更新 ([It's FOSS](#)、[StableBuild](#))。可行做法：使用社群維護的分支 (例如 [pgsty/minio](#))、自行從原始碼建置並固定版本，或改用其他 S3 相容方案。因為程式只透過 S3 API 存取，更換時不需修改程式。

Migrations：第一個檔案 migrations/0001_init.sql 就是「資料模型與 seed」一節的完整結構；第二個檔案放食物分類等固定資料。

規則：已套用的 migration 檔不再修改，任何變更都新增一個檔案；SQL 盡量寫成 SQLite 與 Postgres 都能執行的語法。

設計 tokens (取自 wireframe，寫進 `tailwind.config.js`)

```
theme: {
  extend: {
    colors: {
      ink: '#1F1B1D',          // 主要文字
      muted: '#5E585B',       // 次要文字
      line: '#D9D3CB',        // 分隔線、邊框
      fill: '#EFEFE5',        // 淺底、未選取的分段按鈕
      bg: '#FBFAF7',          // 頁面背景
      surface: '#FFFFFF',     // 卡片
      primary: { DEFAULT: '#A3245F', soft: '#F6E3EC' }, // 主按鈕、選取狀態、蛋白質
      good: { DEFAULT: '#2F6B4C', soft: '#E2F0E7' },    // 完成狀態、碳水
      warm: { DEFAULT: '#8A5A0B', soft: '#F6EBD3' },    // 提醒、脂肪
    },
    fontFamily: {
      display: ['LXGW WenKai TC'], // 標題
      body: ['Noto Sans TC'],      // 內文
    },
    borderRadius: { field: '10px', card: '14px', sheet: '16px' },
  },
}
```

元件規則：可點擊區域至少 44 px；共用元件 FoodOptionRow 以 gluestack-ui 的 Pressable 實作，選取時 `bg-primary-soft`，右側固定 24 px 的 ✓ 位置。

SQLite 的限制與換 Postgres 的時機：同一時間只有一個寫入者（WAL 模式下讀寫可並行），適合個人使用與初期少量使用者。需要多個伺服器實例、大量同時寫入，或使用者分布多個地區時再換 Postgres：SQL 全部集中在 repository 模組，換資料庫時只需調整這一層與少數語法差異（例如日期函式），yoyo-migrations 也支援 Postgres。

開發順序

1. P1 核心：登入、個人資料與目標、今日流程、身形紀錄與圖表、訓練課表編輯、推播提醒
2. P2 餐點：食物分類與食物 seed、自建餐點、等量替換、每日自動分配、主食自動調整
3. P3 AI：拍照辨識與結果確認、Jev 決策層
4. 上架前：隱私權政策、刪除帳號流程、App Store 審核資料

設計決策與待決事項

決策	原因
今日流程一次只顯示一步	專注當下這一步，不用捲動；進度列可回頭修改
餐點由食物分類組成（主食、蛋白質、蔬菜...）	符合實際吃法，方便只換其中一樣
替換時自動等量換算克數	避免同克數換食物造成營養差很多
移除「鎖定」	自動分配改為整道挑選，鎖定不再需要，畫面更單純
欄位名稱用 meal_time 取代 slots	更直白，他人閱讀 API 較容易理解
加入與替換食物共用 FoodOptionRow	兩處操作一致；點整列選取，✓ 位置固定不位移
熱量與營養素只由程式計算	同一份餐點每次數字一致，AI 只負責辨識和選擇
主食份量隨目標自動調整	目標改變時不必逐道修改餐點
資料庫先用 SQLite	開發與部署最簡單、成本最低；SQL 集中在 repository 模組，保留日後換 Postgres 的彈性
後端用 FastAPI（Python）	自動產生 OpenAPI 文件與前端型別；Python 方便處理營養資料匯入與 AI 串接
不使用 ORM，用手寫 SQL + migration 檔	資料表結構和查詢一目了然；migration 是純 SQL，任何人都看得懂也能審查

決策	原因
前端 UI 用 gluestack-ui + NativeWind	元件放在專案裡，容易改成 wireframe 的風格；Tailwind 寫法上手快，設計 tokens 集中管理
內建文字用 translations 表 (name_key)	之後加英文或其他語言只要新增翻譯，不必改資料表；使用者自建的名稱直接存 name
額外食物併入 day_meals 的 extras 時段	早午晚餐與額外食物用同一套結構與 API，少一張表
動作與課表的 kind 改為 category_id	和食物分類的命名與結構一致
自製料理不另外建模，直接用 meal_items	自製料理本質上就是一道餐點的組成，少一張表與一個分類
照片存在 MinIO (S3 相容物件儲存)	照片不佔 API 流量、可獨立擴充與備份；程式只用 S3 API，之後換雲端儲存不需改程式
Google 與 Apple 登入，由 FastAPI 驗證後發自己的 JWT	登入機制維持一套；有第三方登入時 App Store 要求提供 Apple 登入；Google email 已驗證且與現有帳號相同時綁定到同一帳號

待決事項

- ☐ App 先給自己用，還是第一版就上架？決定先做 PWA 或直接做原生 App
- ☐ 推播提醒是否納入第一版
- ☐ 完成畫面是否顯示連續天數（需在 GET /days/{date} 回傳 streak）
- ☐ 第一批 seed 食物清單與營養數字來源確認
- ☐ 圖片模型選擇，以及 Jev 是否在第三階段導入
- ☐ MinIO 官方社群版已停止維護：決定用社群分支、自行建置固定版本，或改用其他 S3 相容方案